

A Fragile Balance of Power

Laboratory of Advanced Programming 2025/2026

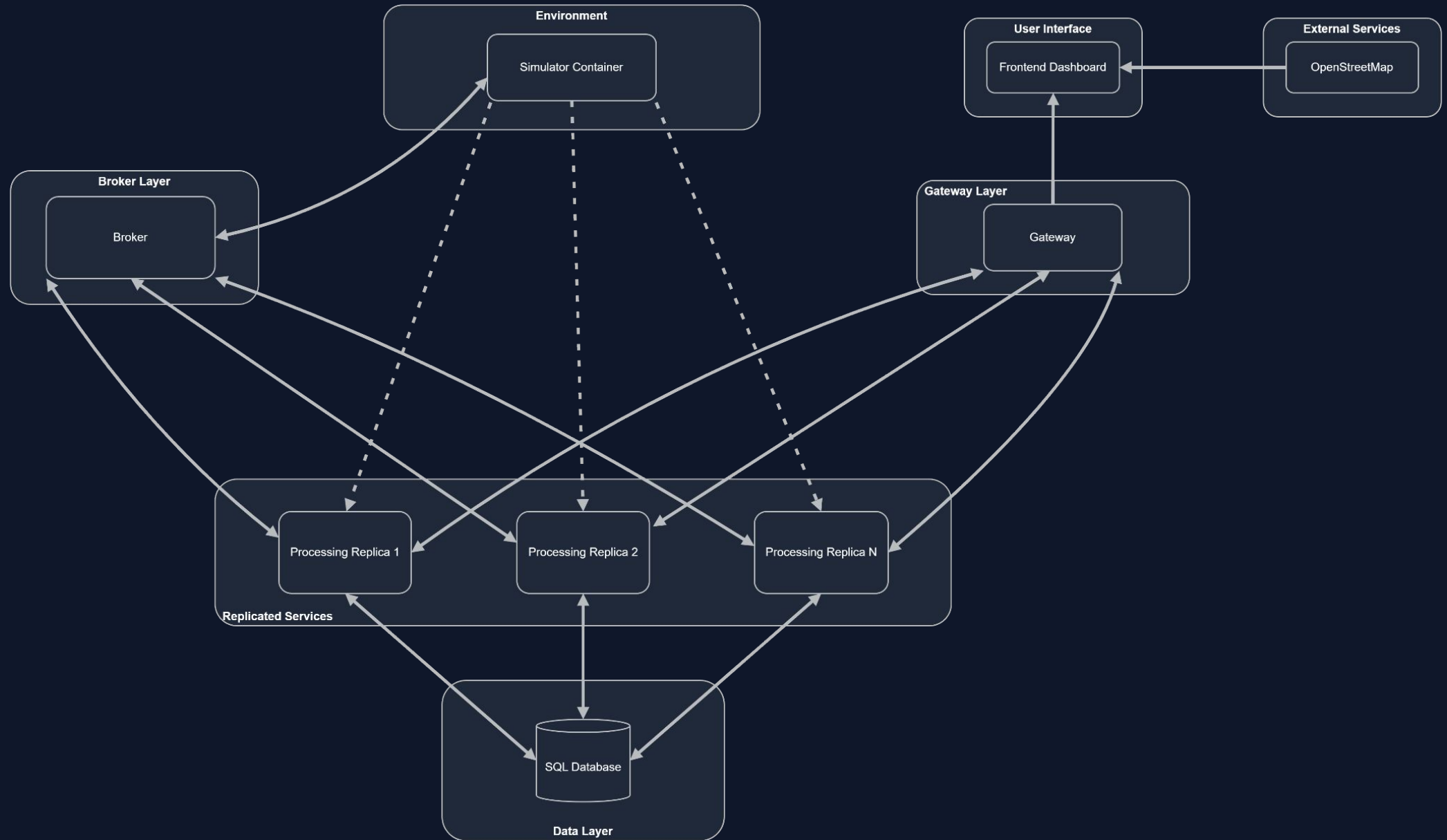
Group: GitPush&Pray

Patriarca Emanuele: 1798796

Patriarca Davide: 1910558

Masi Davide: 1815396

System Architecture



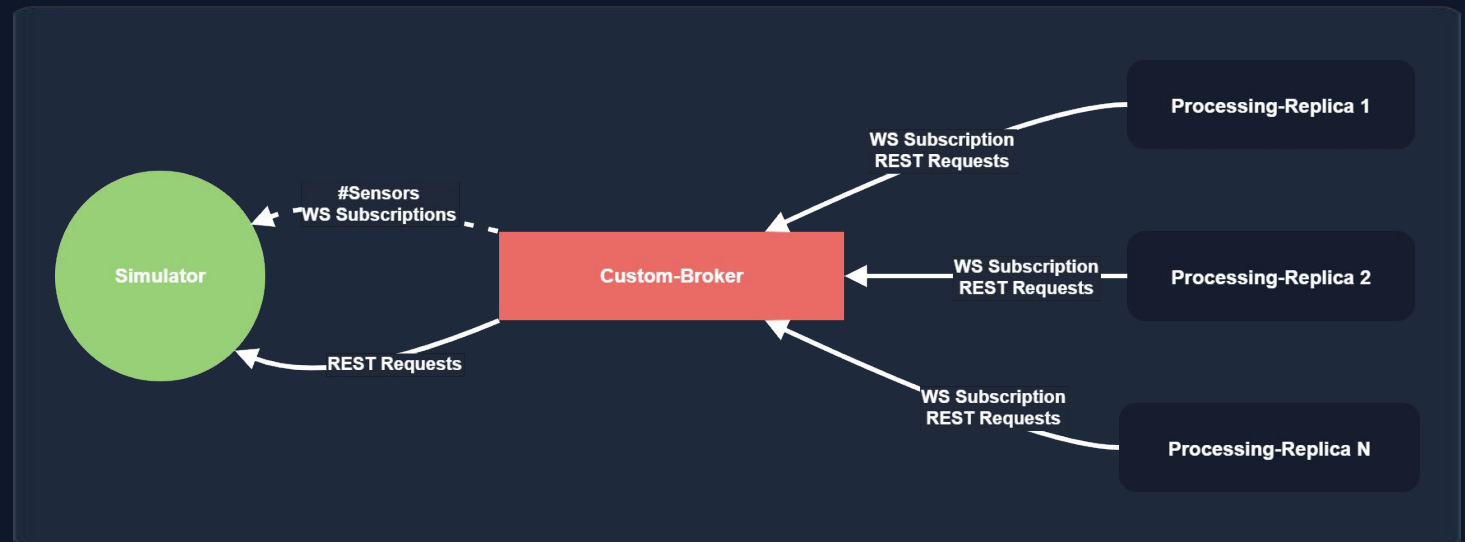
Broker

A lightweight, stateless Message-Oriented Middleware (MOM) designed for high-throughput data ingestion and routing, ensuring zero manipulation of the raw telemetry payloads. It has been developed using FastAPI framework together with websocket and asyncio libraries.

Real-Time Ingestion: Captures continuous 20Hz sensor streams via persistent, asynchronous WebSockets.

Envelope Pattern: Wraps intact raw measurements with a sensor_id tag, acting as a pure data router.

Broadcast Fan-Out: Instantly multiplies and forwards every single data packet to all remote processing replicas.



Processing Service

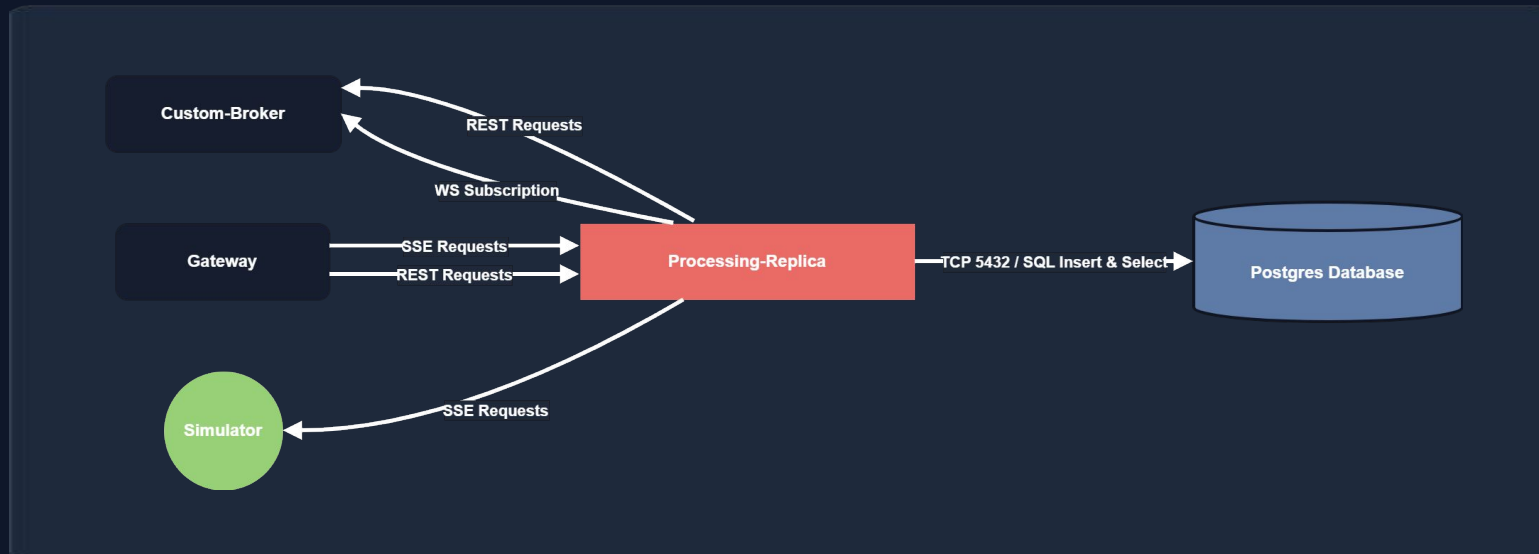
Distributed Analytics & Fault Tolerance

The **Processing Service** is the core analytical engine of the platform. It ingests real-time seismic data from the Custom-Broker, applies Fast Fourier Transform (FFT) on sliding windows to classify threats like earthquakes or nuclear events and makes these events available for the live update of the frontend. It also persists the results to the shared PostgreSQL database.

Shutdown Handling: actively listens for external control commands (SHUTDOWN signals) from the simulator in order to simulate a failure of the service.

Main framework and libraries used:

- FastAPI
- Numpy
- websockets
- asyncio

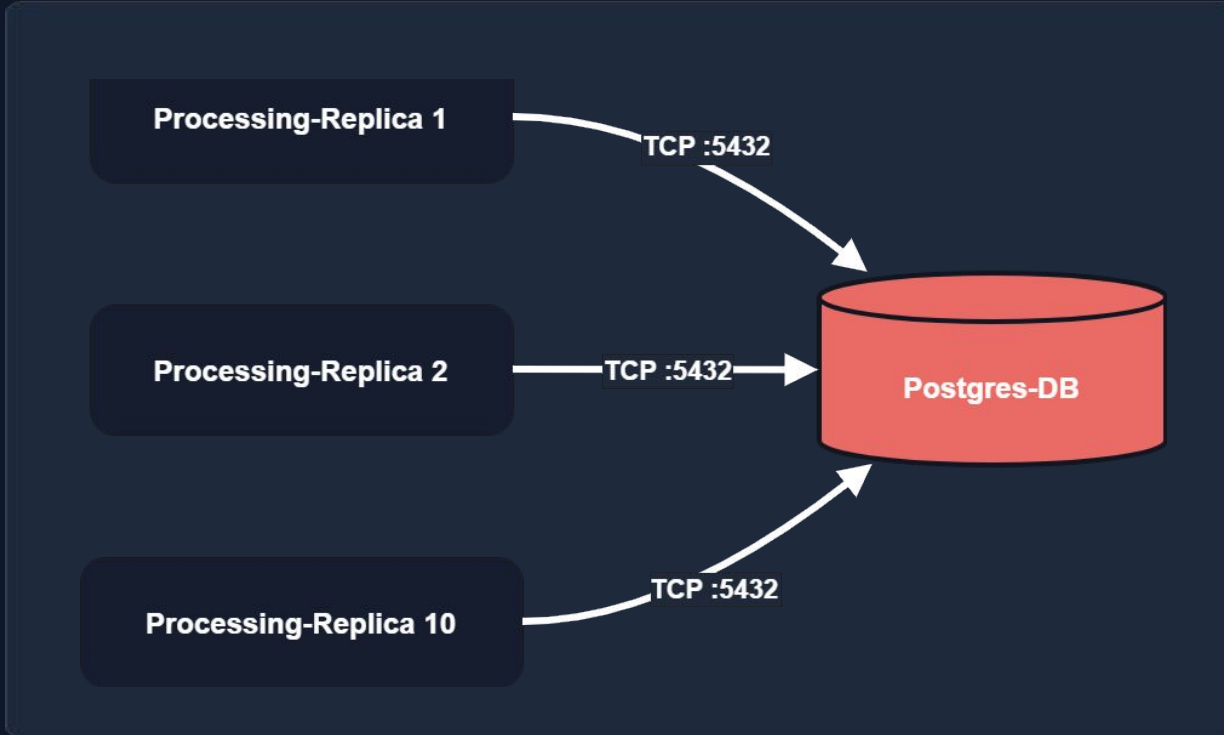


Rules Model

The rule model classifies threats based on the dominant frequency extracted via FFT (Fast Fourier Transform):

$0.5 \leq f < 3.0 \text{ Hz}$	Earthquake
$3.0 \leq f < 8.0 \text{ Hz}$	Conventional Explosion
$f \geq 8.0 \text{ Hz}$	Nuclear-Like event

Database



Preventing Data Duplication

Since our architecture relies on a broadcast model, all 10 replicas receive and process the exact same sensor data simultaneously. This means that if an earthquake occurs, all 10 processing service will try to save it to the database.

We implemented a 'safe-duplication' strategy: first, every replica deterministically assigns an `event_id` to the detected threat for the specific sensor. Then, in our PostgreSQL schema, we established a composite Primary Key made of three fields: `event_id`, `sensor_id`, and `category_event`.

When the replicas attempt their inserts, we use the PostgreSQL `ON CONFLICT DO NOTHING` clause. This makes our insert operations idempotent. The database accepts the first query that arrives, and when the other 9 replicas try to insert the exact same composite key, the database simply ignores them without crashing or throwing errors. This ensures absolute data integrity across the distributed cluster.

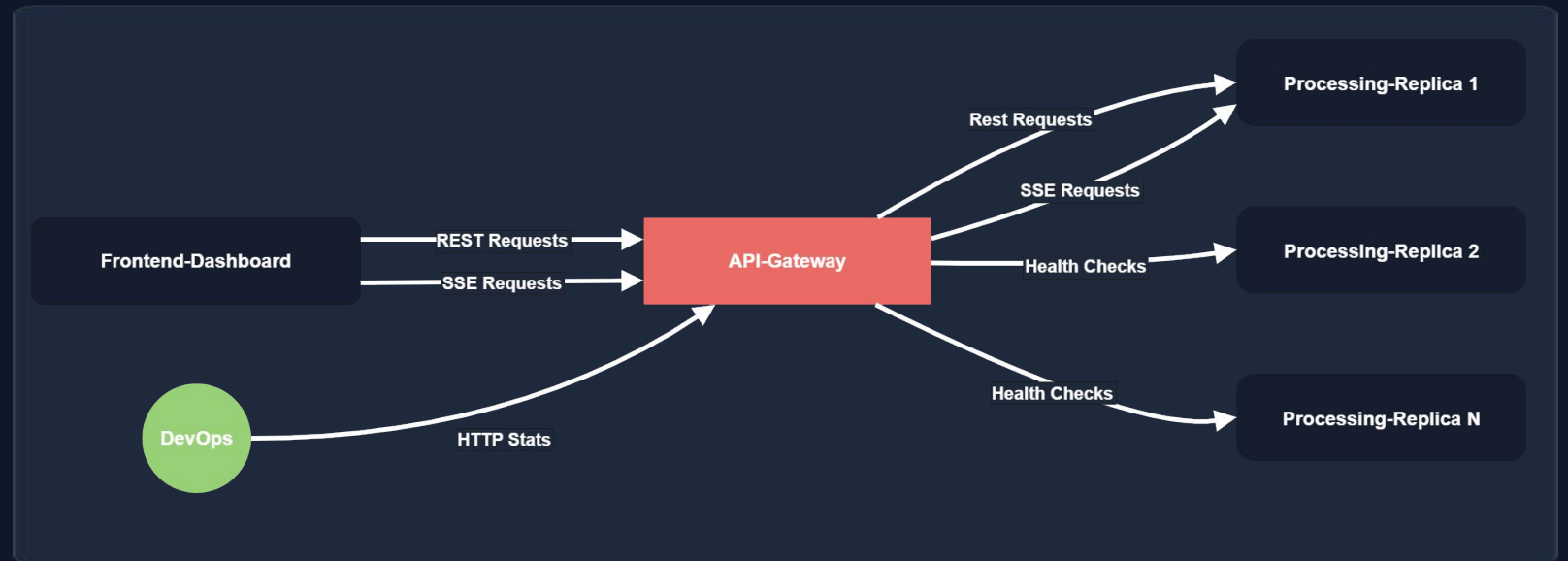
API Gateway

The API Gateway is responsible of providing a single entry point to the frontend while redirecting this unique entry point towards one of the available processing service replica.

The API Gateway has been developed using HAProxy.

Fault Tolerance

- **Instant Detection:** The API Gateway immediately detects the node failure via continuous /health endpoint checks.
- **Automatic Rerouting:** Traffic is seamlessly redirected to healthy replicas.
- **Zero Downtime:** Ensures continuous system availability without any interruption.

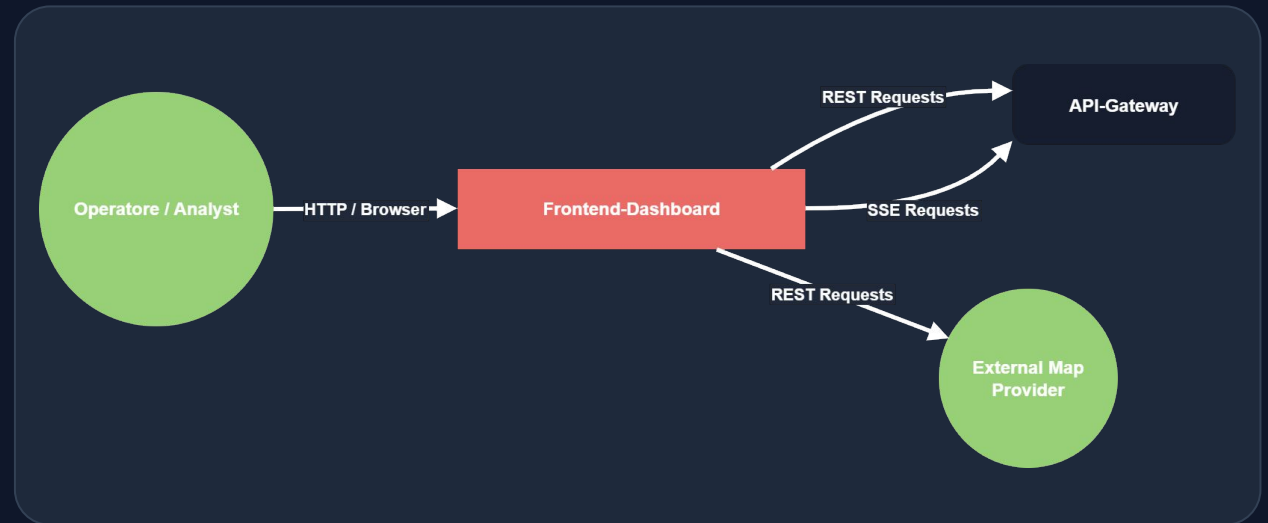


Frontend | Technical Architecture

The frontend is an Angular application (TypeScript) that provides a responsive user interface with interactive dashboards and components for visualizing and managing sensor and seismic event data, styled with Tailwind CSS.

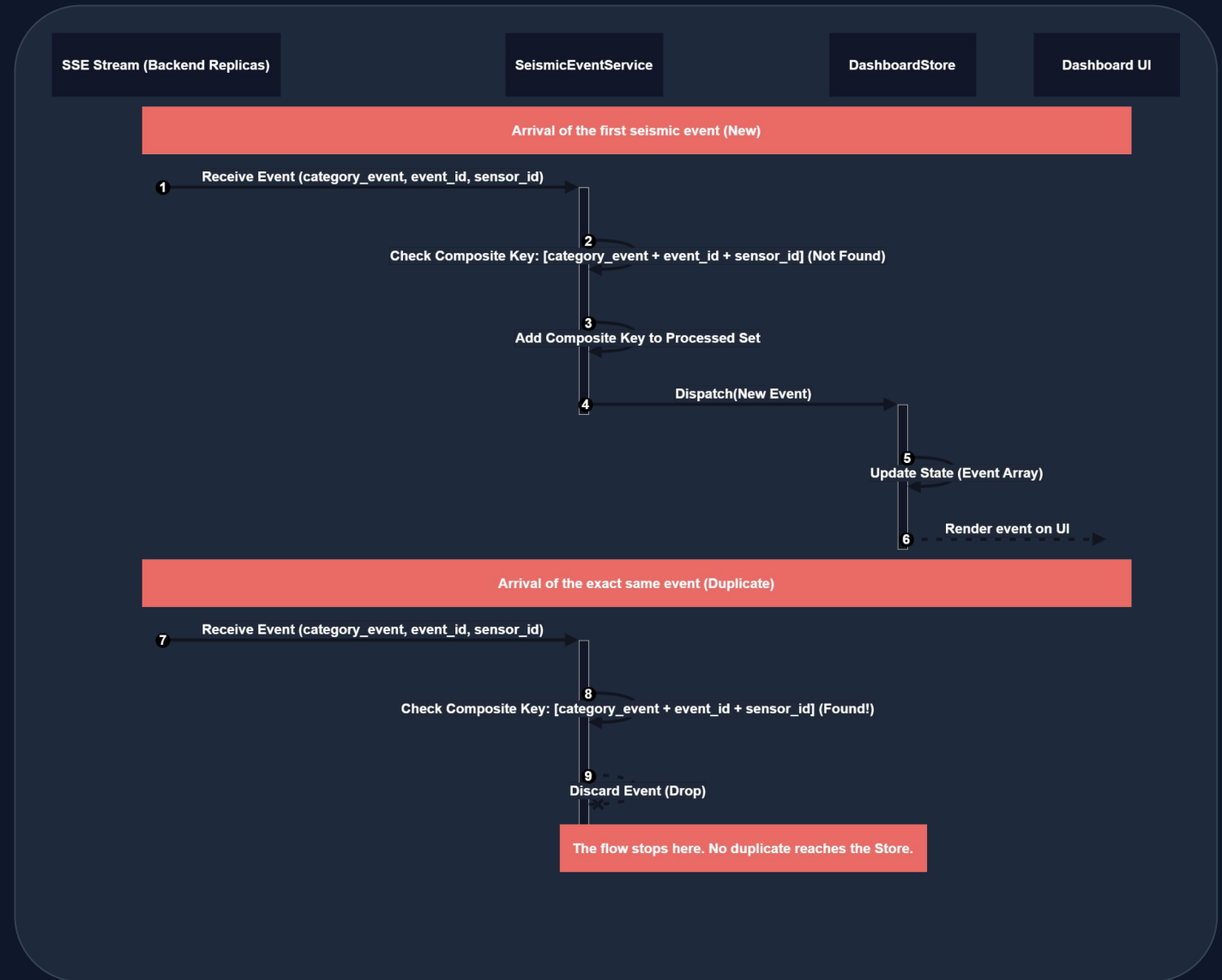
Data Acquisition Mechanisms: Employs dedicated data services (e.g., SeismicEventService) that utilize REST calls for retrieving historical data and active sensor information, and Server-Sent Events (SSE) for real-time streaming of seismic event updates.

Centralized State Management: The DashboardStore provides a centralized mechanism for managing the application's state, integrating all acquired data to ensure consistency and responsiveness across the user interface.



Frontend | Event De-duplication

- **The Composite Key Strategy:** the system guarantees uniqueness of events by evaluating a deterministic composite key for each incoming event. This signature is strictly composed of [category_event + event_id + sensor_id].
- **Collaborative Filtering:** As real-time data arrives via Server-Sent Events (SSE), the SeismicEventService intercepts the payload and cross-references its key against the DashboardStore's active registry of already processed events.
- **Idempotent UI Rendering:** Only genuinely distinct incidents that pass this strict validation are dispatched and committed to the application's observable state. This ensures that duplicate broadcasts from parallel backend replicas are silently dropped, preventing visual clutter and ensuring absolute data accuracy for the end-user.



Presentation concluded.

Thank you for your attention.